

Persistencia con PostgreSQL

Clase 02 — Desarrollo de Software 2026

La misma API de Tareas... pero los datos ya no se pierden

El problema: los datos mueren al reiniciar

Módulo 01 → `tasks: list[dict]` vive en la memoria del proceso

```
uv run main.py    → tareas: ["Aprender HTTP", "Hacer React"]  
Ctrl+C           → 💀 el proceso muere  
uv run main.py    → tareas: [] ← TODO PERDIDO
```

*Los datos que no sobreviven al reinicio **no son datos de verdad**.
Son una demo. Hoy los hacemos reales.*

Mini intro a Docker — ¿cómo conseguimos postgres?

Imagen = el molde. Postgres ya instalado y configurado.

Contenedor = una instancia corriendo de ese molde.

```
docker run -d --name pg-clase02 \
  -e POSTGRES_PASSWORD=postgres \
  -p 5432:5432 \
  postgres:16-alpine

docker ps                # ver contenedores corriendo
docker exec -it pg-clase02 psql -U postgres # entrar a psql
```

Concepto	Analogía
Imagen	Módulo prefabricado: viene con TODO adentro
Contenedor	El módulo enchufado y funcionando
-p 5432:5432	La puerta de entrada (puerto local → puerto del contenedor)

Los alumnos no instalan nada: van directo a **Supabase** (mismo postgres, cero instalación)

Alumnos: Supabase free tier (2 minutos)

1. Crear cuenta en **supabase.com** (plan Free, sin tarjeta)
2. Crear proyecto → **Settings** → **Database** → **Connection string**
3. Copiar la URL del **pooler** (puerto 6543)

```
postgresql://postgres.TU_REF:TU_PASSWORD@aws-0-REGION.pooler.supabase.com:6543/postgres
```

 **Gotcha profesional:** *el free tier pausa el proyecto tras 7 días de inactividad. Los datos **no se pierden** — se reactiva con un click. Esa es la diferencia entre un free tier y producción.*

SQL en 10 minutos — la tabla es una planilla

```
CREATE TABLE IF NOT EXISTS tasks (  
  id          INTEGER      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  title       TEXT         NOT NULL,  
  completed   BOOLEAN      NOT NULL DEFAULT FALSE,  
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()  
);
```

Comando	Qué hace	En nuestras palabras
CREATE TABLE	Define el molde	"La estructura de la planilla"
INSERT	Agrega una fila	"Una tarea nueva"
SELECT	Lee filas	"Listar las tareas"
UPDATE	Modifica filas	"Marcar como hecha"
DELETE	Borra filas	"Eliminar la tarea"

`DEFAULT NOW()` → **la base** genera `created_at` , no el código Python.
`IDENTITY` → **la base** genera el `id` , no Python. La base manda.

La conexión — una URL lo dice todo

```
postgresql:// USUARIO : CONTRASEÑA @ HOST : PUERTO / BDD
^protocolo    ^usuario    ^clave    ^dónde    ^puerta    ^nombre
```

```
load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/postgres")

pool = ConnectionPool(conninfo=DATABASE_URL, min_size=1, max_size=10, open=False)
```

¿Por qué un pool? Abrir una conexión cuesta (handshake TCP + auth + SSL).

El pool mantiene conexiones abiertas y las **reutiliza**:

un estacionamiento con guardia — el auto no fabrica su lugar, el guardia le asigna uno.

Migrar la API — el patrón que se repite

```
with pool.connection() as conn:
    conn.row_factory = dict_row      # fila → {"columna": valor}
    with conn.cursor() as cur:
        cur.execute("... SQL ...", (params,))
        row = cur.fetchone()
```

Pieza	Para qué
<code>with pool.connection()</code>	Pedir conexión prestada → commit automático al salir
<code>dict_row</code>	Leer columnas por nombre: <code>row["title"]</code>
<code>%s</code>	Placeholder — nunca concatenar strings en SQL (inyección)
<code>RETURNING</code>	Postgres devuelve la fila afectada → sin segunda consulta

El mismo endpoint, otro motor

Módulo 01 — Python hacía todo:

```
task = {"id": next_id, "title": body.title.strip(),
        "completed": False, "created_at": datetime.now(timezone.utc).isoformat()}
tasks.append(task)
```

Módulo 02 — la base hace todo:

```
cur.execute("INSERT INTO tasks (title) VALUES (%s) "
            "RETURNING id, title, completed, created_at", (body.title.strip(),))
row = cur.fetchone()
```

El **toggle** también es SQL ahora: `UPDATE tasks SET completed = NOT completed WHERE id = %s`
Y el "no existe" es `WHERE id = %s` devolviendo 0 filas → **404**

✓ Checkpoint 1 — BBDD conectada

```
curl http://localhost:8000/api/health
```

```
{ "status": "Funciona", "service": "02-...", "db": "conectada", "tasks_count": 0 }
```

- ▶ Server arriba, pool abierto, postgres responde
- ▶ Swagger en `http://localhost:8000/docs`
- ▶ Si falla → el problema es la `DATABASE_URL`

El momento de la clase

1. Crear tareas → 2. Matar el servidor → 3. Levantarlo → 4. Listar

```
curl -X POST http://localhost:8000/api/tasks -H "Content-Type: application/json" \
  -d '{"title": "Aprender SQL"}'
# Ctrl+C → uv run main.py → curl http://localhost:8000/api/tasks
```

La tarea sigue ahí. Eso es persistencia.

Y ahora la frutilla: el frontend de la clase 01, sin tocar una línea:

```
cd ../01-mi-primer-app/frontend && npm dev
```

Las tareas aparecen... y sobreviven al reinicio del backend.

El frontend no sabe ni le importa si debajo hay una lista o PostgreSQL.

La API es un contrato entre dos sistemas.

✓ Checkpoint 2 — Persistencia demostrada

- ▶ [] Reinicié el servidor y las tareas siguen
- ▶ [] El frontend del Módulo 01 funciona contra el backend nuevo
- ▶ [] Postman: collection `02-persistencia-postgresql` → flujo feliz + persistencia
- ▶ [] Casos límite: 422 y 404 idénticos al Módulo 01

Resumen

	Módulo 01	Módulo 02
Dónde viven los datos	Lista en memoria	PostgreSQL
¿Sobreviven al reinicio?	✗	✓
¿Quién genera <code>id</code> y <code>created_at</code> ?	Python	La base
¿Quién valida?	Pydantic	Pydantic (la base valida en el próximo nivel)
Frontend	React JS	El mismo, sin cambios

Ejercicios ●●● → en `GUIA_ALUMNO.md`

- ▶ ● Explicá `SELECT 1` del health check
- ▶ ● Agregá `priority` end-to-end (schema → modelo → endpoint)
- ▶ ● Inyección SQL: probá `"; DROP TABLE tasks; --` y explicá por qué `%s` te protege

Hoy los datos se volvieron reales

Clase 03 (próxima): React con TypeScript

"La Universidad te da el mapa. El recorrido lo hacés vos."